SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Particulars	Details
Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration / Mode	3 Days / Individual
Student Reg. No.	192511124
Student Name	Deepak S

SCALABILITY AUDIT OF "SHOPSPHERE"

Design and Analysis of Algorithms – Application-Based Problem Report

Conventions and Assumptions

- **log n** means $\log_2 n$ throughout; recurrences use **T(1) = 1** and n is taken as a power of 2 (so n/2 is always an integer); the general case is obtained with floors/ceilings.
- Servers execute $\mathbf{10^8}$ **basic operations per second**, so the nightly window of 10 minutes (600 s) allows at most $\mathbf{6 \times 10^{10}}$ basic operations.
- Time estimates use the exact operation counts derived below with a unit cost per basic operation. Current size n = 100,000; expected size n = 1,000,000.
- Every formula derived in this report has been verified by brute-force counting in the Python program given in Section 8.

(a) Understanding the Problem

The table identifies, for each module, the input size, the dominating basic operation, where it occurs and the expected growth of its execution count.

Module	Input size	Basic operation	Where it occurs	Expected growth as n grows
M1	n = number of products	Comparison of title and seller of Product[i] with Product[j]	Innermost statement (IF test) executed for every pair (i, j), i < j	Quadratic – one comparison per pair, so doubling n multiplies work by 4
M2	n = upper limit of the window size	Audit(Request[j])	Inner FOR loop, which runs i times while i doubles: 1, 2, 4, …	Linear – the doubling window sizes form a geometric series dominated by its last term

M3	n = number of catalogue / index entries	Comparison of the search key with an entry	S1: each step of the sequential scan. S2: each probe of the middle entry	S1: linear in n. S2: logarithmic – the range halves after every comparison
M4	n = size of the customer–product data	Linear-time merge (combining) work, n units per call, plus the number of sub-problems	The merge step at each node of the recursion tree; 4 (current) or 2 (redesign) sub-problems of size n/2	Current: quadratic. Redesign: n log n
M5	n = number of sales records	Record comparisons made while sorting by revenue	The sort performed at every level of the halving recursion (cost n log n at the top level)	Between n log n and n^2: it grows like n $\log^2$ n
Report gen.	n = number of records	Dominant term of f(n) = $3n^2$ + 10n log n + 50	The $3n^2$ term (nested processing)	Quadratic

Justification

- **M1:** the comparison is the most frequent operation and sits in the innermost loop; everything else (marking a duplicate) occurs at most once per pair and is no more frequent.
- **M2:** the loop control i ← i × 2 runs only about log n times, whereas Audit() is executed inside the inner loop, so Audit() is the dominating operation and the one counted.
- **M3:** the key comparison is the only operation repeated per step; S1 may inspect up to n entries, S2 discards half of the range per comparison.
- **M4 and M5:** these are divide-and-conquer recurrences, so the basic operation is the work done outside the recursive calls (merging in M4, sorting in M5) together with the number of recursive sub-problems.

(b) Analysis of the Iterative Modules

Module M1 – Duplicate-Listing Detector

Basic operation: the comparison "Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller". It is executed once for every pair (i, j) and there is no early exit, so the count does not depend on the data.

For a fixed i, j runs from i + 1 to n − 1, so the inner loop executes (n − 1) − (i + 1) + 1 = n − 1 − i times. Hence

$$C(n) = \Sigma_{i=0}^{n-2} (n - 1 - i) = (n - 1) + (n - 2) + \ldots + 2 + 1$$

$$C(n) = (n - 1) \cdot n / 2 = \mathbf{(n^2 - n) / 2}$$

Time complexity: $C(n) = n^2/2 - n/2 \leq n^2/2$ for $n \geq 1$ and $C(n) \geq n^2/4$ for $n \geq 2$, hence **$\Theta(n^2)$** in the best, average and worst cases.

Space complexity: only the loop variables i, j are used (no extra array), so **$\Theta(1)$** auxiliary space.

Meaning for ShopSphere: n = 100,000 gives 4,999,950,000 ≈ 5×10^9 comparisons (≈ 50 s). When the catalogue grows 10 times to 1,000,000, the work grows 100 times to 499,999,500,000 ≈ 5×10^{11} comparisons, i.e. ≈ 4,999.995 s ≈ **83.3 minutes**.

Module M2 – API Rate Limiter

Basic operation: Audit(Request[j]). The outer loop variable takes the values $i = 2^m$ for m = 0, 1, 2, … as long as $2^m < n$, i.e. $m \leq \lfloor \log n \rfloor - 1$. So the outer loop runs $\lfloor \log n \rfloor$ times. In the pass with $i = 2^m$, the inner loop runs j = 0 … i − 1, that is **$i = 2^m$ times (not n times)**.

$$A(n) = \Sigma_{m=0}^{\lfloor \log n \rfloor - 1} 2^m = 1 + 2 + 4 + \ldots + 2^{\lfloor \log n \rfloor - 1}$$

$$A(n) = \mathbf{2^{\lfloor \log n \rfloor} - 1} \quad \text{(geometric series: } \Sigma\, 2^m = 2^k - 1\text{)}$$

Because $n \leq 2^{\lfloor \log n \rfloor} < 2n$, we obtain **$n - 1 \leq A(n) \leq 2n - 2$**. Therefore A(n) = **Θ(n)**; for n a power of 2, A(n) = n − 1 exactly. The space complexity is **Θ(1)**.

Is the team member's Θ(n log n) claim correct?

No. The claim multiplies "log n passes" by "n operations per pass", but the inner loop does not run n times in every pass; it runs i times, and i is 1, 2, 4, 8, … The last pass contributes about n/2, the one before n/4, and so on, so the sum is a geometric series dominated by its largest term and equals about n, not n log n. O(n log n) is a valid but loose upper bound – it is not tight. The **tight bound is Θ(n)**.

Verification for n = 1,024

Item	Value
Outer-loop values of i	1, 2, 4, 8, 16, 32, 64, 128, 256, 512 ($\lfloor \log_2 1024 \rfloor$ = 10 passes; i = 1024 stops the loop)
Audit operations	1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = **1,023**
Formula $2^{10} - 1$	1,024 − 1 = **1,023 = n − 1** – verified (program output confirms 1023)
Team member's estimate n log n	1,024 × 10 = 10,240 – about **10 times too large**

Meaning for ShopSphere: at n = 1,000,000, A(n) = $2^{20} - 1$ = 1,048,575 operations ≈ 0.0105 s – the rate limiter is cheap and scales linearly.

(c) Solving a Recurrence by Substitution – Search Version S2

Recurrence: **T(n) = T(n/2) + 1, T(1) = 1.** Backward substitution ($n = 2^k$):

$$T(n) = T(n/2) + 1$$

$$T(n/2) = T(n/4) + 1 \qquad T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$$

$$T(n/4) = T(n/8) + 1 \qquad T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3$$

…

After k substitutions: $T(n) = T(n/2^k) + k$

The recursion stops when $n/2^k = 1$, i.e. $2^k = n$, so $k = \log_2 n$. Substituting T(1) = 1:

$$T(n) = T(1) + \log_2 n = \mathbf{1 + \log_2 n}$$

For arbitrary n the exact closed form is **T(n) = $\log_2 n$ + 1**. Check: T(8) = T(4) + 1 = T(2) + 2 = T(1) + 3 = 4 = 1 + $\log_2 8$ (verified); T(1,000,000) = 20 (program output).

Implementation	Time complexity	Space complexity
Iterative binary search	**Θ(log n)**	**Θ(1)** – only low, high, mid are stored
Recursive binary search	**Θ(log n)**	**Θ(log n)** – recursion stack of log n + 1 frames (Θ(1) each)

Meaning for ShopSphere: growing the catalogue from 100,000 to 1,000,000 entries raises the worst-case comparisons of S2 only from 17 to 20. The iterative version is preferable in production because it avoids stack growth.

(d) Applying the Master Theorem

Master Theorem: for T(n) = aT(n/b) + f(n) compare f(n) with $n^{\log_b a}$. **Case 1:** f(n) = $O(n^{\log_b a - \varepsilon})$ T = $\Theta(n^{\log_b a})$. **Case 2:** f(n) = $\Theta(n^{\log_b a})$ T = $\Theta(n^{\log_b a} \log n)$. **Case 3:** f(n) = $\Omega(n^{\log_b a + \varepsilon})$ and $a \cdot f(n/b) \leq c \cdot f(n)$, $c < 1$ T = Θ(f(n)).

Module M4 – Recommendation Engine

Quantity	Current design T(n) = 4T(n/2) + n	Redesign T(n) = 2T(n/2) + n
a	4	2
b	2	2
f(n)	n	n
$n^{\log_b a}$	$n^{\log_2 4} = n^2$	$n^{\log_2 2} = n$
Comparison	$f(n) = n = O(n^{2-\varepsilon})$ with $\varepsilon = 1 > 0$	$f(n) = n = \Theta(n) = \Theta(n^{\log_b a})$
Case used	**Case 1**	**Case 2**
Result	**$T(n) = \Theta(n^2)$**	**$T(n) = \Theta(n \log n)$**
Exact solution (T(1) = 1)	$2n^2 - n$	$n \log_2 n + n$

When the data doubles: for the current design $T(2n)/T(n) \to (2n)^2/n^2 = \mathbf{4}$, i.e. the running time quadruples. For the redesign $T(2n)/T(n) = 2n \cdot \log(2n) / (n \log n) = 2(1 + 1/\log n)$, a little over **2** (the program gives ×2.18 for n = 1,024 → 2,048 and ×4.001 for the current design). At n = 1,000,000 the redesign needs ≈ 2.1×10^7 operations against ≈ 2×10^{12} for the current design – roughly 10^5 times fewer.

Module M5 – Sales Analytics Engine

$T(n) = 2T(n/2) + n \log n$ gives a = 2, b = 2, $f(n) = n \log n$ and $n^{\log_b a} = n$.

- **Case 1** needs $f(n) = O(n^{1-\varepsilon})$: n log n is larger than n, so it fails.
- **Case 2** needs $f(n) = \Theta(n)$: $f(n)/n = \log n \to \infty$ is not a constant, so it fails.
- **Case 3** needs $f(n) = \Omega(n^{1+\varepsilon})$ for some $\varepsilon > 0$: $f(n)/n^{1+\varepsilon} = \log n / n^{\varepsilon} \to 0$ for every $\varepsilon > 0$, so it fails.

Hence **none of the three standard cases applies** – f(n) lies in the polylogarithmic gap between Case 2 and Case 3.

Extended Master Theorem

If $f(n) = \Theta(n^{\log_b a} \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. Here k = 1, so **$T(n) = \Theta(n \log^2 n)$**.

Verification by recursion tree

Level i (i = 0, 1, …, log n − 1) has 2^i nodes of size $n/2^i$, each costing $(n/2^i)\cdot\log(n/2^i)$. The cost of level i is therefore $2^i\cdot(n/2^i)\cdot(\log n - i) = n(\log n - i)$. The n leaves cost $n\cdot T(1) = n$.

$$T(n) = n \sum_{i=0}^{\log n - 1} (\log n - i) + n = n \cdot \log n (\log n + 1) / 2 + n$$

$$T(n) = (n \log^2 n + n \log n)/2 + n = \mathbf{\Theta(n \log^2 n)}$$

Check for n = 1,024 (log n = 10): 1,024 × 10 × 11 / 2 + 1,024 = 57,344, matching the program output. **Meaning:** the analytics engine is only a factor log n slower than an optimal n log n algorithm – acceptable, but improvable (see Section (g)).

(e) Best, Average and Worst Cases

Case	S1 – sequential search (condition)	S1 comparisons	S2 – binary search (condition)	S2 comparisons
Best	Product is the first element	**1**	Key equals the middle entry on the first probe	**1**
Average	Product present, equally likely at any of the n positions	**(n + 1)/2**	Successful search, any entry equally likely	**≈ $\log_2 n - 1$**
Worst	Product is the last element or is absent	**n**	Key absent, or found only at the deepest level	**$\log_2 n$ + 1**

Average case of S1 with probability p that the product exists

If the product is present (probability p), it is equally likely at each position, so a successful search needs (1 + 2 + … + n)/n = (n + 1)/2 comparisons on average. If it is absent (probability 1 − p), all n entries are compared.

$$C_{avg}(n, p) = p \cdot (n + 1)/2 + (1 - p) \cdot n = \mathbf{n(1 - p/2) + p/2}$$

Special cases: p = 1 gives (n + 1)/2; p = 0 gives n. **For n = 1,000,000 and p = 0.5:**

$$C_{avg} = 0.5 \times 1{,}000{,}001 / 2 + 0.5 \times 1{,}000{,}000 = 250{,}000.25 + 500{,}000 = \mathbf{750{,}000.25 \approx 7.5 \times 10^5}$$

comparisons

S2: best case Θ(1), average and worst case Θ(log n) – for n = 1,000,000 the worst case is 20 comparisons and a typical successful search needs about 19 ($\log_2 n - 1 \approx 18.9$). **Meaning:** an average search in S1 costs about 37,500 times more comparisons than in S2 (750,000 versus ≈ 20).

(f) Proving an Asymptotic Bound

Definition: $f(n) = \Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and n_0 such that $c_1\cdot g(n) \leq f(n) \leq c_2\cdot g(n)$ for all $n \geq n_0$. Here $f(n) = 3n^2 + 10n \log_2 n + 50$ and $g(n) = n^2$.

Lower bound

For $n \geq 1$, $10n \log_2 n \geq 0$ and $50 > 0$, so $f(n) \geq 3n^2$. Hence $\mathbf{c_1 = 3}$.

Upper bound

For $n \geq 1$ we have $\log_2 n \leq n$, so $10n \log_2 n \leq 10n^2$; also $50 \leq 50n^2$. Therefore

$$f(n) \leq 3n^2 + 10n^2 + 50n^2 = 63n^2$$

Hence $\mathbf{c_2 = 63}$ and $\mathbf{n_0 = 1}$. Thus $3n^2 \leq f(n) \leq 63n^2$ for all $n \geq 1$, which proves $\mathbf{f(n) = \Theta(n^2)}$. (A tighter pair also works: $c_2 = 4$ with $n_0 = 64$, because at $n = 64$, $10 \cdot 64 \cdot 6 + 50 = 3{,}890 \leq 4{,}096 = 64^2$, and $n/\log n$ keeps increasing afterwards. Both pairs were checked numerically by the program.)

f(n) is not O(n)

Assume, for contradiction, that $f(n) \leq c \cdot n$ for some constant $c > 0$ and all $n \geq n_0$. Since $f(n) \geq 3n^2$, this gives $3n^2 \leq c \cdot n$, i.e. $n \leq c/3$, for every $n \geq n_0$. But choosing $n = \max(n_0, \lceil c/3 \rceil + 1)$ violates $n \leq c/3$ – a contradiction. Equivalently, $f(n)/n \geq 3n \to \infty$, so no constant c can bound it. Hence $\mathbf{f(n) \neq O(n)}$.

Meaning: the report generator grows quadratically; at $n = 1{,}000{,}000$ it performs $\approx 3 \times 10^{12}$ operations ≈ 8.3 hours, far beyond a nightly window.

(g) Scalability Decision

Estimates for $n = 1{,}000{,}000$ at 10^8 operations/second; nightly window = 10 min = 600 s = 6×10^{10} operations.

Module	Basic operations	Running time	Within 10 min?	Verdict
M1 ($\Theta(n^2)$)	5.0×10^{11}	≈ 83.3 min (4,999.995 s)	**No**	**Redesign**
M2 ($\Theta(n)$)	1.05×10^6	≈ 0.0105 s	Yes	Keep
S2 ($\Theta(\log n)$)	20	$\approx 2 \times 10^{-7}$ s	Yes	Keep
M4 current ($\Theta(n^2)$)	2.0×10^{12}	≈ 20,000 s ≈ 5.56 h	**No**	**Redesign**
M4 redesign ($\Theta(n \log n)$)	2.09×10^7	≈ 0.21 s	Yes	Adopt
M5 ($\Theta(n \log^2 n)$)	2.10×10^8	≈ 2.1 s	Yes	Acceptable
Report generator ($\Theta(n^2)$)	3.0×10^{12}	≈ 8.33 h	**No**	Redesign

Modules that fail the 10-minute window: M1 (≈ 83 min), the current M4 (≈ 5.6 h) and also the nightly report generator (≈ 8.3 h). All three have $\Theta(n^2)$ cost.

Recommendations

- **M1 – must be redesigned.** Replace the all-pairs comparison with a hash set keyed on (seller, title): one pass over the data, expected $\mathbf{\Theta(n)}$ time and $\Theta(n)$ extra space, $\approx 10^6$ operations ≈ 0.01 s. A deterministic alternative is to sort the products by (seller, title) in $\Theta(n \log n)$ (≈ 0.2 s) and mark equal adjacent records. This trades memory for a reduction from 83 minutes to well under a second.

- **M4 – must be redesigned.** Adopt the proposed design without replicated partitions: the Master Theorem moves from Case 1, $\Theta(n^2)$, to Case 2, $\Theta(n \log n)$; running time falls from ≈ 5.6 hours to ≈ 0.2 seconds and memory duplication is removed.
- **Report generator – should be redesigned** (e.g. by removing its quadratic nested processing using hashing or sorting), because $\Theta(n^2) = 3 \times 10^{12}$ operations exceeds the window.
- **M5 – acceptable, optional improvement.** At ≈ 2.1 s it meets the window, but sorting once at the start ($\Theta(n \log n)$) instead of at every recursion level would remove a log factor.
- **M2 and S2 – no change needed.** M2 is linear (≈ 0.01 s) and S2 needs only about 20 comparisons per query. S1 (average ≈ 7.5×10^5 comparisons) should not be used for large catalogues; S2 on a sorted index is the correct choice.

Source Code and Output Verification

The program below implements M1, M2, S1, S2, M4, M5 and f(n) as counting routines, compares the counts with the derived closed forms and reproduces the scalability table. File: **Source_Code/shopsphere_audit.py**.

Source code – shopsphere_audit.py

```
"""CSA06 - CO1 Assessment Tool 1: Scalability Audit of ShopSphere
Verifies every derivation in the report by brute-force counting.
Author: Deepak S (192511124)"""
import math

OPS_PER_SEC = 10**8
WINDOW_SEC = 10 * 60

# ---------- M1: duplicate detector ----------
def m1_count(n):
    c = 0
    for i in range(0, n - 1):
        for j in range(i + 1, n):
            c += 1                  # one title+seller comparison
    return c

# ---------- M2: API rate limiter ----------
def m2_count(n):
    c, i = 0, 1
    while i < n:
        for j in range(i):
            c += 1                  # Audit(Request[j])
        i *= 2
    return c

# ---------- M3: S2 recurrence and real binary search ----------
def t_s2(n):
    return 1 if n <= 1 else t_s2(n // 2) + 1

def binary_search_worst(n):
    arr = list(range(n)); worst = 0
    for key in list(range(n)) + [-1, n + 5]:
        lo, hi, comps = 0, n - 1, 0
        while lo <= hi:
            mid = (lo + hi) // 2; comps += 1
            if arr[mid] == key: break
            lo, hi = (mid + 1, hi) if arr[mid] < key else (lo, mid - 1)
        worst = max(worst, comps)
    return worst

# ---------- M4 / M5 recurrences (T(1)=1, n = power of 2) ----------
def t_m4_cur(n): return 1 if n == 1 else 4 * t_m4_cur(n // 2) + n
def t_m4_new(n): return 1 if n == 1 else 2 * t_m4_new(n // 2) + n
def t_m5(n):     return 1 if n == 1 else 2 * t_m5(n // 2) + n * math.log2(n)

def fmt_time(s):
    if s < 1e-3: return f"{s:.2e} s"
    if s < 60:   return f"{s:.3f} s"
    if s < 3600: return f"{s/60:.2f} min"
    return f"{s/3600:.2f} h"

print("=== M1: comparisons = n(n-1)/2 ===")
for n in (10, 100, 1000):
    print(f"n={n:5d}  counted={m1_count(n):8d}  formula={n*(n-1)//2:8d}")

print("\n=== M2: Audit operations (claim: Theta(n log n); derived: 2^ceil(log2 n) - 1) ===")
```

```
for n in (16, 100, 1024, 10**6):
    f = 2 ** math.ceil(math.log2(n)) - 1
    print(f"n={n:8d}  counted={m2_count(n):9d}  formula={f:9d}  n log2 n={n*math.log2(n):12.0f}")

print("\n=== S2: T(n)=T(n/2)+1 -> floor(log2 n)+1 ===")
for n in (1, 2, 8, 1000, 1024, 10**6):
    print(f"n={n:8d}  recurrence={t_s2(n):3d}  formula={int(math.log2(n))+1:3d}")
print("Worst case of a real binary search, n=1023 ->", binary_search_worst(1023), "(expected 10)")

print("\n=== S1 average comparisons: C(n,p) = p(n+1)/2 + n(1-p) ===")
n, p = 10**6, 0.5
print(f"n={n}, p={p}  ->  {p*(n+1)/2 + n*(1-p):.2f} comparisons")

print("\n=== M4 and M5 closed forms (T(1)=1) ===")
for n in (2**4, 2**10):
    k = int(math.log2(n))
    print(f"n={n:5d} M4 cur={t_m4_cur(n):8d} (2n^2-n={2*n*n-n:8d}) | "
          f"M4 new={t_m4_new(n):6d} (n log n+n={n*k+n:6d}) | "
          f"M5={t_m5(n):10.1f} (n k(k+1)/2+n={n*k*(k+1)/2+n:10.1f})")
print("Doubling n=2^10 -> 2^11:  M4 current x%.3f   M4 redesign x%.3f"
      % (t_m4_cur(2**11)/t_m4_cur(2**10), t_m4_new(2**11)/t_m4_new(2**10)))

print("\n=== f(n) = 3n^2 + 10 n log2 n + 50 :  3n^2 <= f(n) <= 63 n^2 for n >= 1 ===")
f = lambda n: 3*n*n + 10*n*math.log2(n) + 50
ok1 = all(3*n*n <= f(n) <= 63*n*n for n in range(1, 20001))
ok2 = all(f(n) <= 4*n*n for n in range(64, 20001))
print("c1=3, c2=63, n0=1 holds for n=1..20000:", ok1)
print("c1=3, c2=4,  n0=64 holds for n=64..20000:", ok2)
print("f(n)/n at n=10^3, 10^5, 10^7:", [round(f(x)/x) for x in (10**3, 10**5, 10**7)], "(unbounded => not O(n))")

print("\n=== Scalability at n = 1,000,000 (10^8 ops/s, window = 10 min = 6x10^10 ops) ===")
n = 10**6; k = math.log2(n)
rows = [
    ("M1 Duplicate detector  n(n-1)/2",   n*(n-1)/2),
    ("M2 Rate limiter        2^20 - 1",   2**20 - 1),
    ("S2 Binary search       floor(lg n)+1", int(k) + 1),
    ("M4 current   2n^2 - n",             2*n*n - n),
    ("M4 redesign  n lg n + n",           n*k + n),
    ("M5 Analytics n lg n (lg n+1)/2 + n", n*k*(k+1)/2 + n),
    ("Report generator f(n)",             f(n)),
]
for name, ops in rows:
    t = ops / OPS_PER_SEC
    print(f"{name:40s} ops={ops:14.4e}  time={fmt_time(t):>10s}  {'FAIL' if t > WINDOW_SEC else 'PASS'}")
```

Program output

```
=== M1: comparisons = n(n-1)/2 ===
n=   10  counted=     45  formula=     45
n=  100  counted=   4950  formula=   4950
n= 1000  counted=  499500  formula=  499500

=== M2: Audit operations (claim: Theta(n log n); derived: 2^ceil(log2 n) - 1) ===
n=     16  counted=       15  formula=       15  n log2 n=         64
n=    100  counted=      127  formula=      127  n log2 n=        664
n=   1024  counted=     1023  formula=     1023  n log2 n=      10240
n= 1000000  counted=  1048575  formula=  1048575  n log2 n=   19931569

=== S2: T(n)=T(n/2)+1 -> floor(log2 n)+1 ===
n=       1  recurrence=  1  formula=  1
n=       2  recurrence=  2  formula=  2
n=       8  recurrence=  4  formula=  4
n=    1000  recurrence= 10  formula= 10
n=    1024  recurrence= 11  formula= 11
n= 1000000  recurrence= 20  formula= 20
Worst case of a real binary search, n=1023 -> 10 (expected 10)
```

```
=== S1 average comparisons: C(n,p) = p(n+1)/2 + n(1-p) ===
n=1000000, p=0.5  ->  750000.25 comparisons

=== M4 and M5 closed forms (T(1)=1) ===
n=   16 M4 cur=     496 (2n^2-n=     496) | M4 new=    80 (n log n+n=    80) | M5=     176.0 (n k(k+1)/2+n=     176.0)
n= 1024 M4 cur= 2096128 (2n^2-n= 2096128) | M4 new= 11264 (n log n+n= 11264) | M5=   57344.0 (n k(k+1)/2+n=   57344.0)
Doubling n=2^10 -> 2^11:  M4 current x4.001   M4 redesign x2.182

=== f(n) = 3n^2 + 10 n log2 n + 50 :  3n^2 <= f(n) <= 63 n^2 for n >= 1 ===
c1=3, c2=63, n0=1 holds for n=1..20000: True
c1=3, c2=4,  n0=64 holds for n=64..20000: True
f(n)/n at n=10^3, 10^5, 10^7: [3100, 300166, 30000233] (unbounded => not O(n))

=== Scalability at n = 1,000,000 (10^8 ops/s, window = 10 min = 6x10^10 ops) ===
M1 Duplicate detector  n(n-1)/2          ops=   5.0000e+11  time=   1.39 h  FAIL
M2 Rate limiter        2^20 - 1          ops=   1.0486e+06  time=  0.010 s  PASS
S2 Binary search       floor(lg n)+1     ops=   2.0000e+01  time=2.00e-07 s  PASS
M4 current   2n^2 - n                    ops=   2.0000e+12  time=   5.56 h  FAIL
M4 redesign  n lg n + n                  ops=   2.0932e+07  time=  0.209 s  PASS
M5 Analytics n lg n (lg n+1)/2 + n       ops=   2.0960e+08  time=  2.096 s  PASS
Report generator f(n)                    ops=   3.0002e+12  time=   8.33 h  FAIL
```

Analysis of the output: the counted values equal the derived formulas in every case – M1 = $n(n-1)/2$, M2 = $2^{\log n} - 1$ (1,023 for n = 1,024), S2 = $\log n + 1$, M4 = $2n^2 - n$ and $n \log n + n$, M5 = $n \cdot k(k+1)/2 + n$ – and the scalability table agrees with Section (g).

Conclusion

Module	Complexity	Time at $n = 10^6$	Decision
M1	$\Theta(n^2)$	≈ 83.3 min	Redesign (hashing / sorting)
M2	$\Theta(n)$ – not $\Theta(n \log n)$	≈ 0.01 s	Keep
M3 – S1 / S2	$\Theta(n)$ / $\Theta(\log n)$	≈ 0.01 s / 2×10^{-7} s	Use S2
M4 current / redesign	$\Theta(n^2)$ / $\Theta(n \log n)$	≈ 5.56 h / 0.21 s	Adopt redesign
M5	$\Theta(n \log^2 n)$	≈ 2.1 s	Acceptable
Report generator	$\Theta(n^2)$	≈ 8.33 h	Redesign

The audit shows that only the quadratic components (M1, the current M4 and the report generator) threaten the nightly window once the catalogue reaches 1,000,000 products; the remaining modules scale comfortably.

Code of Conduct

I **Deepak S (192511124)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____